

Remédiation algorithmique et programmation (Programmation impérative en C++) Cours 3 : Vecteurs et Lecture de fichiers

K.P. Ametana

Supports de cours largement empruntés à Pierre Pesneau
`komlanvi-parfait.ametana@math.u-bordeaux.fr`

Université de Bordeaux
Bât A33 - Bur 265

Sommaire

Les vecteurs à une dimension

Les tableaux à deux dimensions (et plus)

Lecture/Ecriture dans un fichier

Paramètres de la fonction main

Les vecteurs (C++)

Ce cours présente la classe (généralisation d'un type) `vector` de la STL (Standard Template Library).

Ils permettent de manipuler des tableaux à une dimension (un seul indice).

Ils ne correspondent pas au type tableau de base défini dans le langage C/C++.

Un vecteur (tableau à une dimension) est une structure permettant de stocker plusieurs éléments d'un même type. Chaque élément est référencé par un indice.

Déclaration

Un objet (variable) de la classe (type) `vector` est un tableau à une dimension.

Syntaxe :

```
#include<vector>
...
vector<type> nom_vec;
vector<type> nom_vec(taille);
vector<type> nom_vec(taille,init);
```

- **type** : type des valeurs stockées ;
- **nom_vec** : identifiant du vecteur ;
- **taille** : taille du vecteur ;
- **init** : valeur d'initialisation.

Contrairement aux tableaux C/C++, si vous ne précisez pas de valeur d'initialisation, les éléments du vecteur sont initialisés à une valeur par défaut. Cependant je vous conseille fortement de prendre l'habitude d'initialiser vos vecteurs.

Accès

L'accès à un élément d'un vecteur se fait par l'opérateur d'accès direct `[]` en indiquant l'indice de l'élément entre les crochets.

Attention !

Les indices d'un vecteur de taille n vont de 0 à $n - 1$!

```
#include<vector>
...
int taille;
vector<int> vec1(20);
vector<double> tab(15,5.0);

cout << "Entrer une taille" << endl;
cin >> taille;
vector<int> v(taille);

cin >> v[5];
v[3] = 10;
v[0] = v[3] - 5;
cout << v[0] << endl;
```

Méthodes utiles

Un objet d'une classe possède des méthodes (fonctions) permettant de le manipuler.

La méthode `size()` d'un vecteur permet de récupérer sa taille.

Exemple :

```
vector<int> vec(10,3);

cout << ``taille de vec : `` << vec.size() << endl;

for (int i=0; i < (int)vec.size(); i++){
    cout << ``Entrer l'element `` << i << `` du vecteur : `` << endl;
    cin >> vec[i];
}

for (int i=0; i < (int)vec.size(); i++){
    cout << vec[i] << `` ``;
}
cout << endl;
```

Méthodes utiles (suite)

Les méthodes `resize(taille)` et `resize(taille,init)` d'un vecteur permettent de redimensionner le vecteur.

- Si la taille augmente, elles rajoutent des éléments (éventuellement initialisés à la valeur `init`) à la fin du vecteur ;
- Si la taille diminue, elles suppriment des éléments à la fin du vecteur ;
- S'ils ne sont pas supprimés, les valeurs déjà existantes sont conservées.

Exemple :

```
vector<int> vec(10,3);  
cout << ``taille de vec : `` << vec.size() << endl; //affiche 10  
  
vec.resize(15);  
cout << ``taille de vec : `` << vec.size() << endl; //affiche 15
```

Méthodes utiles (suite)

La méthode **push_back(val)** permet de rajouter un élément de valeur **val** à la fin du vecteur. La taille du vecteur augmente de 1.

Exemple :

```
vector<int> vec;
int n;
cout << "Combien d'élément désirez-vous entrer ? " << endl;
cin >> n;

for (int i=0; i<n; i++){
    int val;
    cout << "Entrez une valeur" << endl;
    cin >> val;
    vec.push_back(val);
}
```

La méthode **pop_back()** permet de supprimer le dernier élément du vecteur. La taille du vecteur diminue de 1.

Sommaire

Les vecteurs à une dimension

Les tableaux à deux dimensions (et plus)

Lecture/Ecriture dans un fichier

Paramètres de la fonction main

Les tableaux à deux dimensions (deux indices)

En C/C++, il n'existe pas de tableaux ou de vecteurs à deux dimensions à proprement parler.

Pour représenter un tableau à deux dimensions (une matrice), on utilise un vecteur de vecteurs.

vecteur

vecteur de vecteurs

Vecteur de vecteurs

Déclaration (et initialisation) :

```
int m,n;
cout << "Nombre de lignes ?" << endl;
cin >> m;
cout << "Nombre de colonnes ?" << endl;
cin >> n;
vector< vector<double> > vec(m, vector<double>(n,0.0));
```

Autre possibilité :

```
int m,n;
cout << "Nombre de lignes ?" << endl;
cin >> m;
cout << "Nombre de colonnes ?" << endl;
cin >> n;
vector< vector<double> > vec(m);
for (int i=0; i<vec.size(); i++){
    vec[i].resize(n,0.0);
}
```

Vecteur de vecteurs

Accès :

```
vector< vector<int> > vec(m, vector<int>(n,0.0));
```

- v est un vecteur de vecteurs (d'entiers);
- $v[i]$ est un vecteur (d'entiers);
- $v[i][j]$ est un élément (du type entier) de la matrice.

Attention : un langage de programmation n'a pas la notion de lignes ou de colonnes dans un tableau. C'est simplement la convention mathématique qui suppose que le premier indice correspond à l'indice de ligne et le second à l'indice de colonne.

Tableau à plus de deux dimensions

Pour déclarer des tableaux à plus de deux dimensions, il suffit d'imbriquer les vecteurs : un tableau à trois dimensions est un vecteur de tableau à deux dimensions, c'est-à-dire, un vecteur de vecteurs de vecteurs.

Sommaire

Les vecteurs à une dimension

Les tableaux à deux dimensions (et plus)

Lecture/Ecriture dans un fichier

Paramètres de la fonction main

Utilité

Les fichiers permettent de :

- sauvegarder des données entre deux exécutions d'un programme ;
- récupérer les données provenant d'une autre source ;
- transmettre les données à une autre application.

La lecture et l'écriture dans un fichier est très similaire à la lecture au clavier et à l'affichage à l'écran.

Écriture dans un fichier

`cout` : flot de sortie sur la sortie standard (écran).

Pour écrire dans un fichier, il suffit de créer un flot de sortie redirigé sur le fichier.

- `#include <fstream>` : bibliothèque contenant la définition des types flots sur fichier.
- `ofstream` : type flot de sortie sur un fichier.
- `open(nom_fic)` : méthode de la classe `ofstream` permettant d'associer le fichier `nom_fic` au flot sur lequel la méthode est appelée.
- `close()` : méthode de la classe `ofstream` permettant de fermer le fichier (assure la bonne sauvegarde des données).

Ecriture dans un fichier

Exemple :

```
#include<iostream>
#include<fstream> // utilisation de flot sur des fichiers
#include<string>

using namespace std;

int main(){

    string filename;
    cout << "Entrez le nom du fichier" << endl;
    cin >> filename; // Saisie du nom du fichier dans lequel écrire

    ofstream outfile(filename); // déclaration et ouverture combinée du fichier
    ...
    int x = 10;
    outfile << x << endl; // ecriture dans le flot de données associés au fichier
                          // l'écriture ce fait comme l'écriture à l'écran
                          // en remplaçant le "cout" par le flot créé (ici "outfile")
    ...
    outfile.close(); // fermeture du fichier : assure la bonne sauvegarde du fichier.

    ...
    return 0;
}
```

Ouverture d'un fichier en écriture

- Le répertoire où est ouvert le fichier, est celui où est exécuté votre programme.
- Vous pouvez spécifier un autre répertoire en intégrant le chemin (relatif ou absolu) du fichier dans le nom de votre fichier.
- Si le fichier n'existe pas, il est créé.
- Si le fichier existe, il est supprimé et remplacé par un nouveau fichier.

Lecture dans un fichier

`cin` : flot en entrée depuis l'entrée standard (clavier).

Pour lire dans un fichier, il suffit de créer un flot en entrée redirigé depuis le fichier.

- `#include <fstream>` : bibliothèque contenant la définition des types flots sur fichier.
- `ifstream` : type flot en entrée sur un fichier.
- `open(nom_fic)` : méthode de la classe `ifstream` permettant d'associer le fichier `nom_fic` au flot sur lequel la méthode est appelée.
- `close()` : méthode de la classe `ifstream` permettant de fermer le fichier.

Lecture dans un fichier

- La lecture dans un fichier se fait “mot” par “mot” avec les espaces et les retours à la ligne comme délimiteurs.
- La méthode `eof()` renvoie un booléen à `true` si on a atteint la fin du fichier, et `false` sinon.
- Le répertoire où est ouvert le fichier, est celui où est exécuté votre programme.
- Vous pouvez spécifier un autre répertoire en intégrant le chemin (relatif ou absolu) du fichier dans le nom de votre fichier.
- Si le fichier n'existe pas, il n'est pas créé. Vous devez vérifier que l'ouverture s'est bien passée (voir exemple).

Lecture dans un fichier

Un exemple de lecture :

```
#include<iostream>
#include<fstream> // utilisation de flot sur des fichiers
#include<string>

using namespace std;

int main(){
    string filename;
    ... // dont saisie du nom du fichier

    ifstream infile(filename); // déclaration et ouverture du flot en entrée
    if (infile) {                // Test de la bonne ouverture du fichier
        ...
        while (!infile.eof()){
            string tmp;
            infile >> tmp >> ws; // lecture dans le flot de données associé au fichier.
                                   // ws permet de s'assurer que le flot en entrée va sauter les espaces
                                   // et retour à la ligne entre deux informations pertinentes à lire.
        }
        ...
        infile.close();          // fermeture du fichier
    } else {                     // L'ouverture à échouée -> message d'erreur
        cout << "Erreur : fichier inexistant" << endl;
        abort();
    }
    return 0;
}
```

Remarques

Quand le fichier dans lequel on écrit est destiné à être lu par un programme, on s'assure :

- de structurer le fichier afin de permettre une lecture facile ;
- d'indiquer suffisamment d'informations toujours pour faciliter la lecture.

Sommaire

Les vecteurs à une dimension

Les tableaux à deux dimensions (et plus)

Lecture/Ecriture dans un fichier

Paramètres de la fonction main

Commande linux

Syntaxe classique d'une commande Unix :

`cmd [-opt1] [-opt2] ... [--] arg1 arg2 ...`

- `cmd` correspond au nom de la commande ;
- `-opt` correspond à une option possible (les crochets `[]`, indiquant que c'est une option, n'apparaissent pas lors de l'écriture de la commande ;
- `arg` correspond à un argument obligatoire.

Exemple :

`cp toto.txt tata.txt`

Pour que vos programmes puissent prendre des arguments, il faut déclarer des paramètres à la fonction `main`

Paramètre de la fonction main

La fonction main peut prendre deux paramètres :

```
int main (int argc, char** argv){
```

- argc contient le nombre d'arguments donné au programme (nom du programme compris) ;
- argv est un tableau (version native du C/C++) contenant des chaînes de caractères (version native du C/C++).

Ainsi :

- argv[0] contient le nom du programme ;
- argv[1] contient le premier argument ;
- argv[2] contient le deuxième argument ;
- ...

Exemple d'utilisation

On peut utiliser les arguments d'un programme pour lui faire passer le nom d'un fichier contenant des données.

Si votre programme `monprog.cpp` est du style :

```
#include <iostream>
#include <fstream>

using namespace std;

int main(int argc, char** argv){
    if (argc == 1){
        cout << "veuillez donner un nom de fichier en premier argument" << endl;
        abort(); // Permet d'arrêter violemment le programme
    }

    string filename = argv[1]; // On suppose que le premier argument contient le nom du fichier.
    cout << "Lecture du fichier " << filename << endl;
    ifstream infile(filename);

    ... // Lecture des données du fichier

    infile.close();

    return 0;
}
```

Exemple d'utilisation suite

Si votre fichier de données s'appelle `data.txt`
et qu'après compilation votre executable s'appelle `monprog`

Vour lancerez votre programme avec la commande

```
monprog data.txt
```